



Order Book Analyzer – Status Report 1

FINTECH 512 - Software Engineering for FinTech

Full team of five members for the group project (section 1)



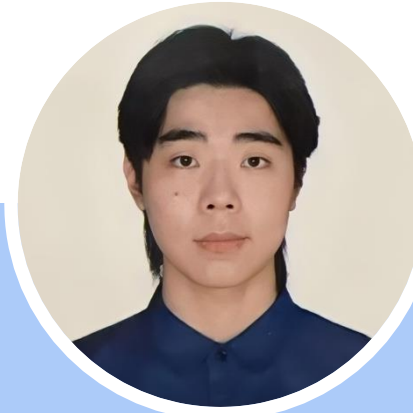
Rebeca Hechtman
NetId: rh398



Mike Lin
NetId: hl561



Mory Shi
NetId: cs836



Loki Luo
NetId: lz294



Alan Song
NetId: ys501

Agenda

- 01.** Project Ideation
- 02.** GitLab Repository and Issues
- 03.** Initial Design
- 04.** Status, Review and Planning

Project Ideation

Project's 'elevator pitch'



Bloomberg of Prediction Markets



Prediction markets, where people bet real money on the outcomes of future events (like whether Bitcoin will hit \$110,000 this year), are exploding in popularity. But making sense of all that betting activity can be frustrating. The raw data behind these markets is messy, disorganized, and hard to easily analyze.

Order Book Analyzer cuts through that noise. Think of us as the Bloomberg for prediction markets. We take massive amounts of complicated, second-by-second betting data and translate it into clear, powerful dashboards and insights. Instead of wrestling with messy data, our platform gives you the exact trends and historical information you need to see exactly where the smart money is going.

Project Scope Disclaimer: This project will be focused and dedicated exclusively to Polymarket BTC 15-min Up/Down (Phase 1). Future market expansions remain under evaluation based on technical complexity and timeline constraints.

The Lean Canvas

Designed for:

Order Book Analyzer

Designed by:

Team Hotel

Date:

03/16/2026

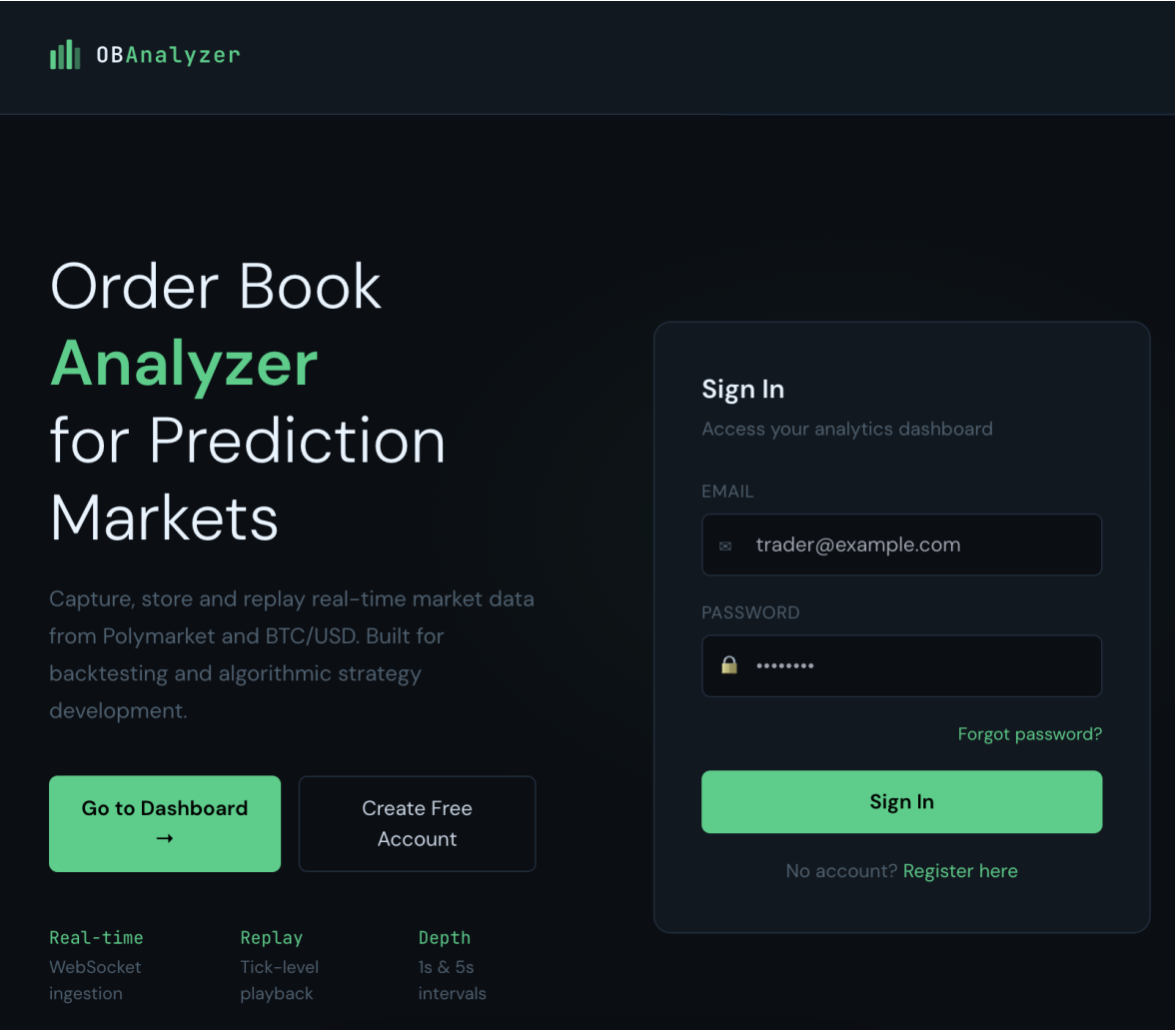
Version:

2.0

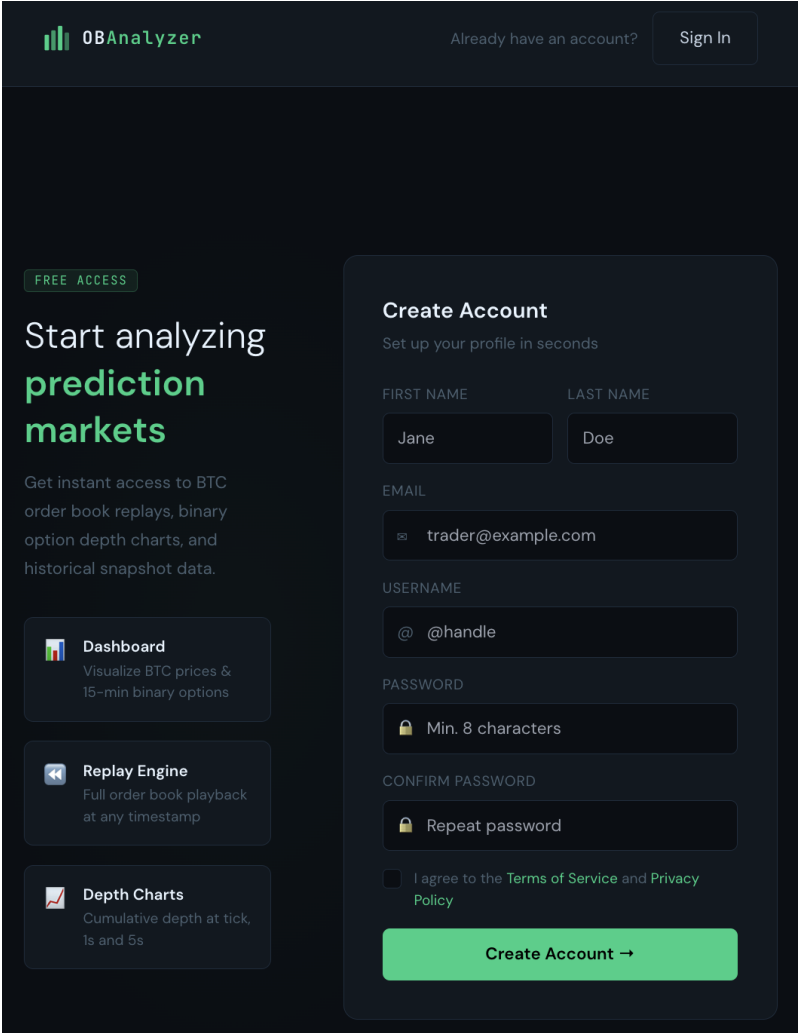
Problem  1. Access & Opacity: High-frequency data (L2/L3 order books) is notoriously difficult to access, painful data wrangling and gaps. 2. Capacity to Process: Normalizing raw prediction market streams into usable research formats requires specialized engineering effort. 3. Signal-to-Noise Ratio: Traditional financial data is too noisy for pure behavioral signal extraction.	Solution  A website provides Insights & Dashboards: Insights, dashboard, and KPI for one or more popular prediction markets trades (Phase 1: Polymarket's BTC 15-min Up/Down, Phase 2: Expand prediction asset classes)	Unique Value Prop.  High quality, tick-level historical data and analytics for high-frequency prediction markets, starting with ultra-short-term crypto volatility (15-minute Up/Down from Polymarket)	Unfair Advantage  1. Curated Historical Data Archive: Because live data is fleeting and heavy to store, reconstructing this past data is nearly impossible for new entrants. 2. Domain & Technical Expertise	Customer Segments  1. Mid and small size Hedge Funds 2. Sophisticated algo traders who wants to trade in prediction market
Existing Alternatives  1. Eagle Alpha 2. Dune Analytics 3. DIY engineering 4. Basic Polymarket Dashboards	Key Metrics  Growth: 1. # of users 2. Retention 3. API call volumes 4. Daily Active User System Health: 1. Data Quality Score	High-Level Concept  1. Yahoo Finance for Prediction Market 2. Bloomberg for Prediction Market	Channels  1. Social Media (LinkedIn, X, Reddit, Red Note) 2. Organic Growth 3. Search Engine 4. Ads	Early Adopters  Crypto-native quants and algo traders currently hitting the limitations of Polymarket's visual dashboards, specifically those running high-frequency arbitrage or momentum strategies on BTC short-term derivative markets.
Cost Structure  Data Infrastructure: Heavy cloud compute and storage costs (AWS EC2/S3, hosting specialized streaming and database clusters) to maintain gapless WebSocket ingestion. Variable costs for data query and usage. Engineering & People: Time and resources dedicated to pipeline maintenance, API integration, and handling schema changes from the source markets. Customer Acquisition & Marketing: Sales outreach, conference attendance, and content creation. Compliance: Legal frameworks and terms of service management.			Revenue Streams  Revenue Model: Freemium <u>Freemium Model:</u> Basic access to delayed visualizations or aggregated dashboards to drive top-of-funnel user growth. <u>Key Account (KA) Annual Licenses:</u> High-ticket, recurring institutional subscriptions with base data transfer quotas for academic labs and funds. <u>Value-Added Services (Overage):</u> Usage-based billing for massive historical downloads that exceed base quotas, or premium access to specialized datasets.	

Prototype Screens

1- Home Page

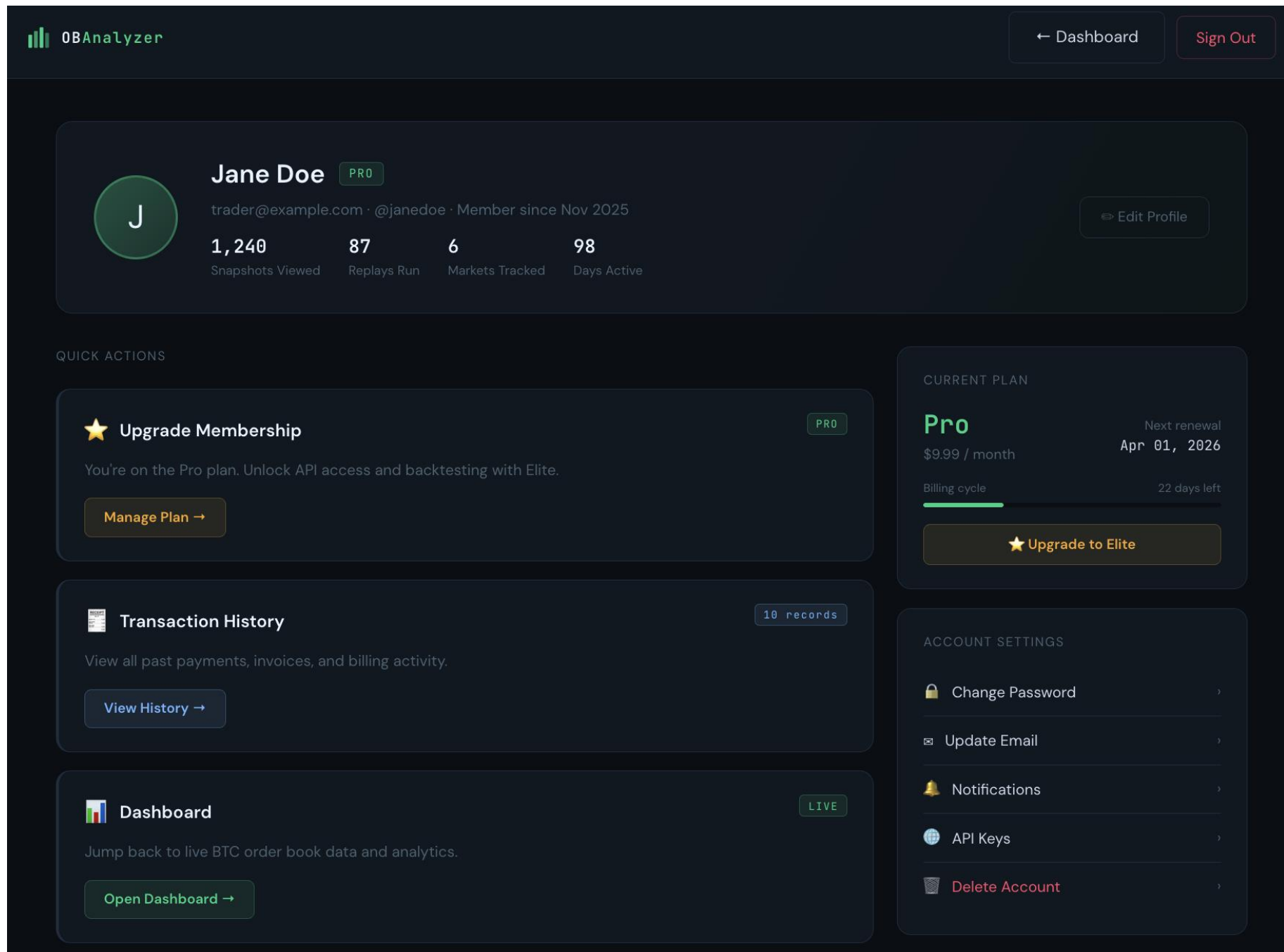


2- New User



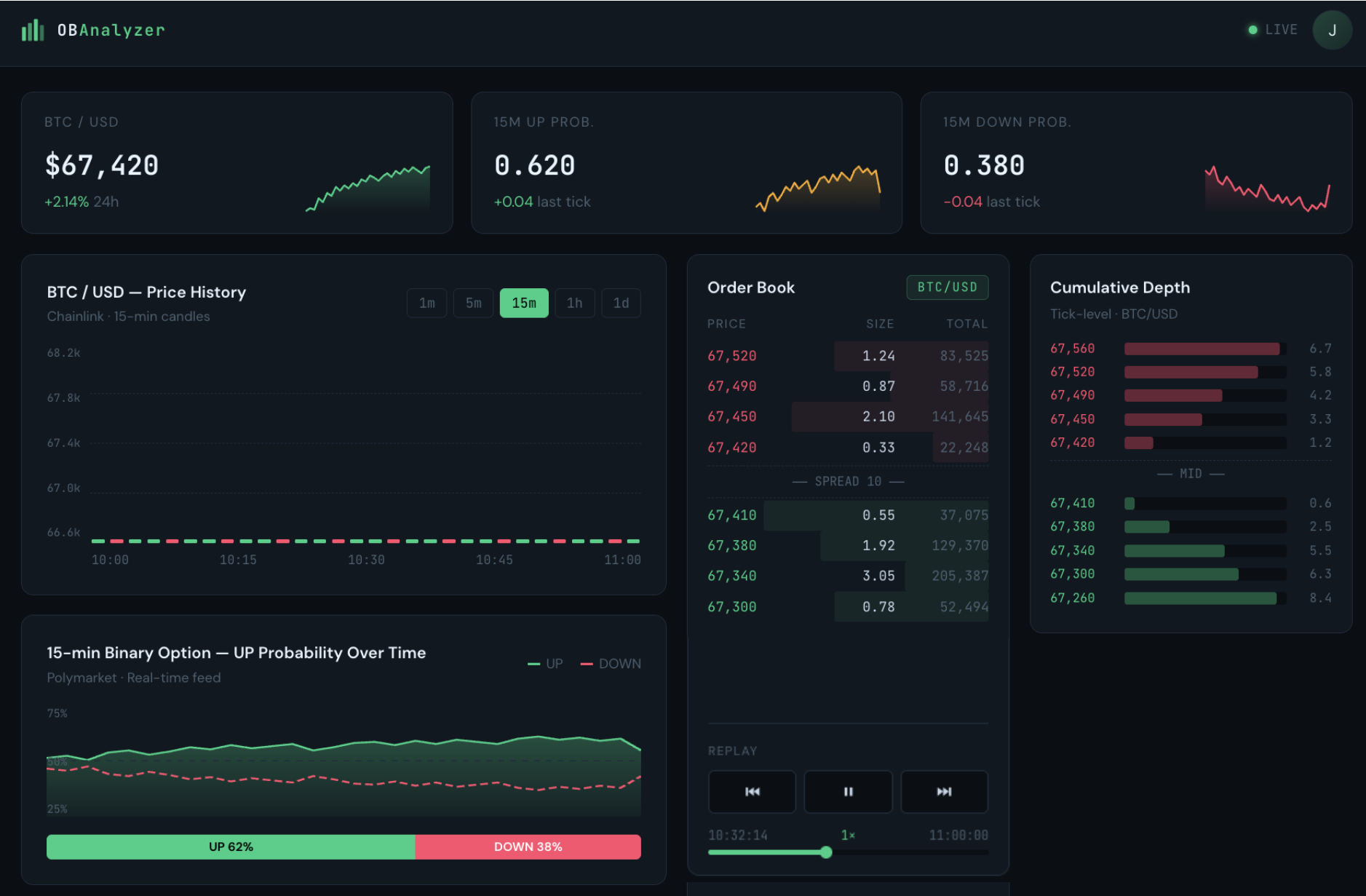
Prototype Screens

3- Profile



Prototype Screens

4-Dashboard



GitLab Repository and Issues

Team Project GitLab Repository

https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#

The screenshot shows a web browser displaying the GitLab interface for a repository named 'pred_market_data_platform' under the 'team-hotel' namespace. The URL in the address bar is 'gitlab.oit.duke.edu/team-hotel/pred_market_data_platform#'. The left sidebar contains a 'Project' section with a list of items: 'pred_market_data_platform', 'Pinned', 'Issues', and 'Merge requests'. Below this is a 'Manage' section with links for 'Plan', 'Code', 'Build', 'Secure', 'Deploy', 'Operate', 'Monitor', 'Analyze', and 'Settings'. At the bottom of the sidebar are links for 'What's new' and 'Help'. The main content area shows the repository name 'pred_market_data_platform' with a 'Code' button. Below this is a message stating 'The repository for this project is empty' and instructions to clone or upload files. There are sections for 'Command line instructions', 'Configure your Git identity' (with 'Local' and 'Global' tabs), 'Git local setup' (with instructions and a code block for configuring git), 'Add files' (with 'SSH' and 'HTTPS' tabs), and 'Create a new repository' (with a code block for cloning). On the right side, there is a 'Project information' section with an 'Invite your team' button and a list of actions: 'Upload File', 'New file', 'Add README', 'Add LICENSE', 'Add CHANGELOG', 'Add CONTRIBUTING', 'Set up CI/CD', 'Add Wiki', and 'Configure Integrations'. At the bottom right, it says 'Created on March 06, 2026'.

team-hotel / pred_market_data_platform

Search or go to...

Project

- pred_market_data_platform
- Pinned
- Issues
- Merge requests

Manage

- Plan
- Code
- Build
- Secure
- Deploy
- Operate
- Monitor
- Analyze
- Settings

What's new

Help

Collapse sidebar

team-hotel / pred_market_data_platform

pred_market_data_platform

Code

The repository for this project is empty

To get started, clone the repository or upload some files.

Command line instructions

You can also upload existing files from your computer using the instructions below.

Configure your Git identity

Get started with Git and learn how to configure it.

Local Global

Git local setup

Configure your Git identity locally to use it only for this project:

```
git config --local user.name "rebecca.hechtman@duke.edu"
git config --local user.email "rh398@duke.edu"
```

Add files

Push files to this repository using SSH or HTTPS. If you're unsure, we recommend SSH.

SSH HTTPS

How to use SSH keys?

Create a new repository

```
git clone git@gitlab.oit.duke.edu:team-hotel/pred_market_data_platform.git
```

Project information

Invite your team

Add members to this project and start collaborating with your team.

Invite members

Upload File

- New file
- Add README
- Add LICENSE
- Add CHANGELOG
- Add CONTRIBUTING
- Set up CI/CD
- Add Wiki
- Configure Integrations

Created on

March 06, 2026

Project Issues

https://gitlab.oit.duke.edu/team-hotel/pred_market_data_platform/-/issues

Closed:

- #1: Create Lean Canvas document
- #2: Develop paper prototype screens
- #3: Set up Team GitLab Repository
- #4: Define initial database schema
- #5: Draft UML class diagram

Open/In-Progress:

- Architecture Documentation (C4 Model)
- Security Threat Modeling
- Containerization
- System Architecture Setup
- Logging & Monitoring
- OAuth2 Authentication
- CI/CD Pipeline
- Full Website UI Design
- Data Dashboard Page
- Homepage UI Prototype
- Market Analytics Charts
- Historical Data Query Layer
- Data Storage Layer
- Polymarket API Integration
- Website Deployment on VM
- User Account System

Link to your project requirements document

<https://duke.box.com/s/b9j6rdc5zg165nhuga59pv1w49vvsqz5>

Major Requirements:

1. Main Functionality Requirements:

- Enable customers to interact with the website. Specifically, allow customers to create accounts, view data and analytics, and subscribe to the website.
- Maintain running instance of the website, running on a virtual machine to continuously pull data from polymarket and display on the website.
- Enable customers to check their account history and activities.
- Enable users to create new account for themselves. Tests available for developers of the team and TAs

2. Users Requirements:

- Add use cases for new user to access the website, subscribe to the website, view the data, charts, analytics, and check their account.
- Write test cases for checking 2 different types of historical data. e.g.
 - Check bitcoin up market data at 15 minutes duration, by tick, timestamp, December 9, 10:00AM
 - Check bitcoin down market data at 15 minutes, by 1 second buckets, timestamp, December 12, 10:00AM
- Parameters:
 - Market ID: Series number for different types of market
 - Book playback granularity: Duration of the buckets of displayed trades
 - Timestamp: the period of the data which the user wants to investigate.

3. Data Integration Requirements:

- The system shall integrate external data from at least one third-party REST/JSON API (e.g. Polymarket).
- Retrieve data programmatically via HTTP requests.
- Parse and validate JSON responses.
- Persist relevant data in the system database.
- Handle API errors, rate limits, and invalid responses gracefully.

4. Charting Requirements:

- Include the strike price, resolve price, and price at selection
- Include chain-link BTC/USD over time price chart
- Include Best Bid, Best Ask, Spread data
- Include a Mid Price (Snapshot view) over time chart
- Include a size over price chart
- Include cumulative depth over price chart
- Include the trader summary history
- Include a section of loaded data

5. UI/UX Design Focus - Prototyping I: Paper Prototype (Home page/Main page)

- Develop a Home page prototype that tells user what this website is about and display our prop.
- Develop a general data page prototype which will be where the main data charts reside. This will be the core function of this website. It will include all the charts mentioned above, allowing users to pick parameters of their desire and print the corresponding data and chart on the screen with already-calculated analytical indicators.

6. UI/UX Design Focus - Prototyping II: Raw webpages

- Fill in the design of other pages based on the style and core implementation of the main and home pages
- Come up with the overall design scheme, such as background color, font, icon, name, and viewports.

7. UI/UX Design Focus - Prototyping III: Final Polish

- Gather feedback from other peers and see what to improve.
- Make sure the overall style is coherent and consistent
- Make sure it is easy to use, and nothing is hard to see or find.

Project Specific Requirements:

1. System Architecture

- Data Ingestion (Hot Path):
 - Python WebSocket client to ingest market data (PolyMarket).
 - Redis cache for real-time data access.
 - REST/WS strategy execution service for trade decisions.
- Transform Layer
 - Kafka message broker with multiple topics.
 - Debezium for CDC from OLTP database.
 - Python Pub/Sub services for processing and routing events.
- Analytics & Deploy
 - Strategy and backtesting services (Jupyter, Streamlit).
 - Object-oriented strategy design.
 - Containerized deployment (Docker, Docker Compose).
 - CI/CD integration with cloud container registry.
- Persistence (Off Path)
 - OLTP database: PostgreSQL.
 - OLAP database: ClickHouse.

2. Implementation of CI/CD pipeline

- Builds the application on every commit.
- Runs the full automated test suite.
- Executes GitLab SAST scanning.

- Deploys the application to the target VM/container environment upon successful build.

3. Security Requirements

- Threat Modeling:
 - Create a Data Flow Diagram (DFD).
 - Identify assets, actors, and trust boundaries.
 - Perform STRIDE analysis on the system
- Static Analysis
 - Run semgrep on the codebase.
 - Address or document identified vulnerabilities.

4. Architecture Documentation (C4 Model)

- System Context Diagram
- Container Diagram
- Component Diagram
- Code-Level Diagram

5. Logging & Monitoring

- Follow OWASP Logging Cheat Sheet guidelines.
- Define:
 - Events to log
 - Log storage location
 - Log format (structured logging required)
 - Sensitive data handling
- Document in `log_design.md`
- Log security-relevant events (auth, failures, data access, trade execution).
- Ensure logs support incident investigation.
- Prevent log tampering and unauthorized access.

6. OAuth2 Integration Requirements

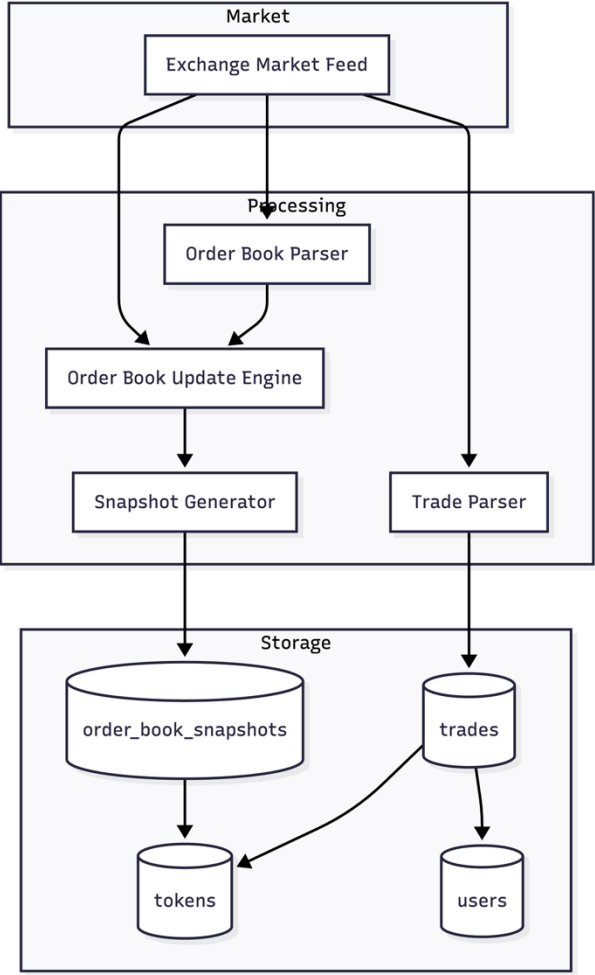
- Integrate a third-party OAuth2 provider (e.g., Google).
- Allow users to authenticate via OAuth2 login.
- Securely store and manage access tokens.
- Protect authenticated routes and restrict access to authorized users only.
- Ensure no user passwords are stored locally.

7. Containerization Requirements

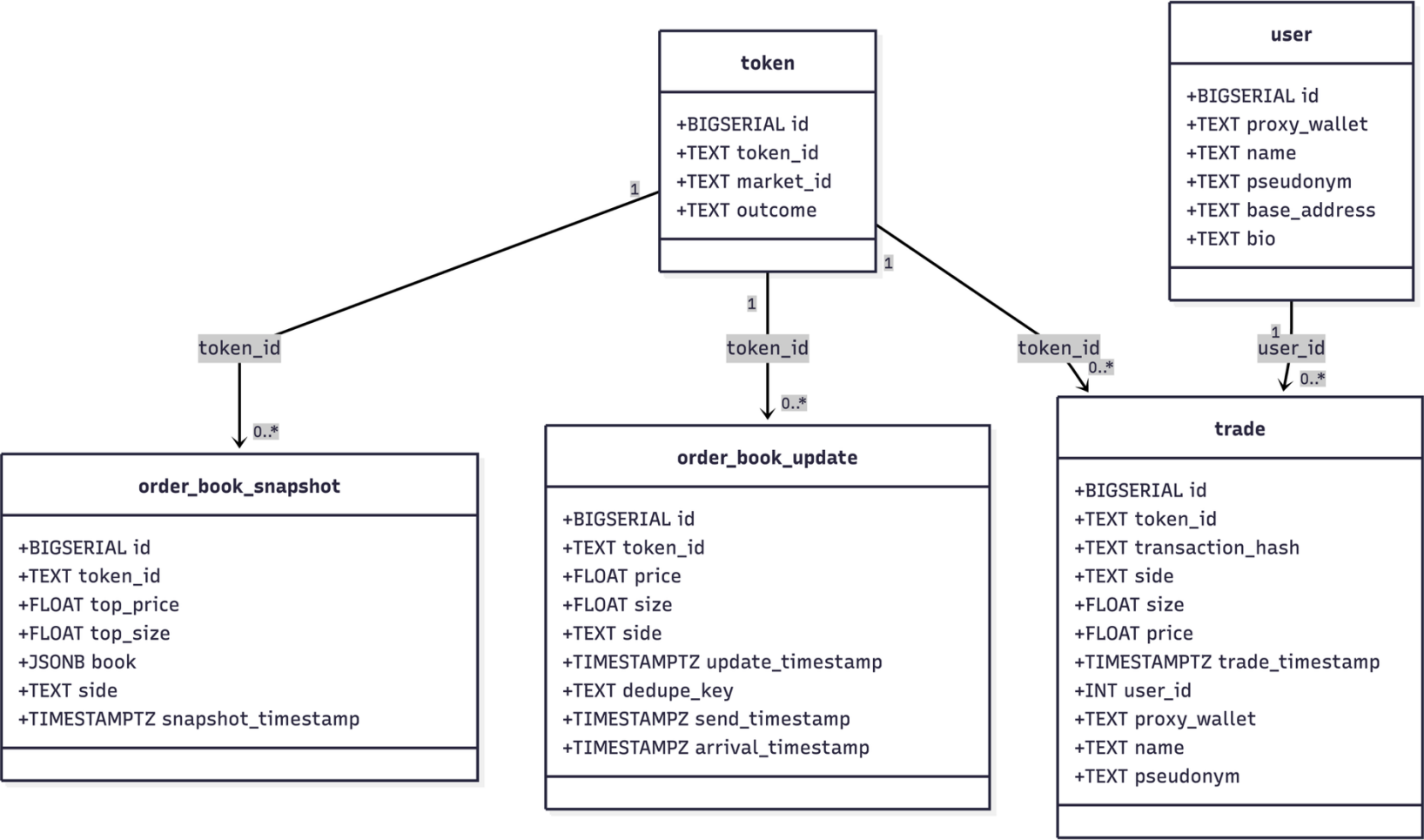
- Provide a Dockerfile that builds a production-ready image of the BigBucks application.
- Provide a docker-compose.yml file that:
 - Deploys the application and required services (e.g., database).
 - Creates persistent volumes for database storage.
- Use Docker Compose to deploy and run the application on the target virtual machine.

Initial Design

UML Class Diagram – Order Book

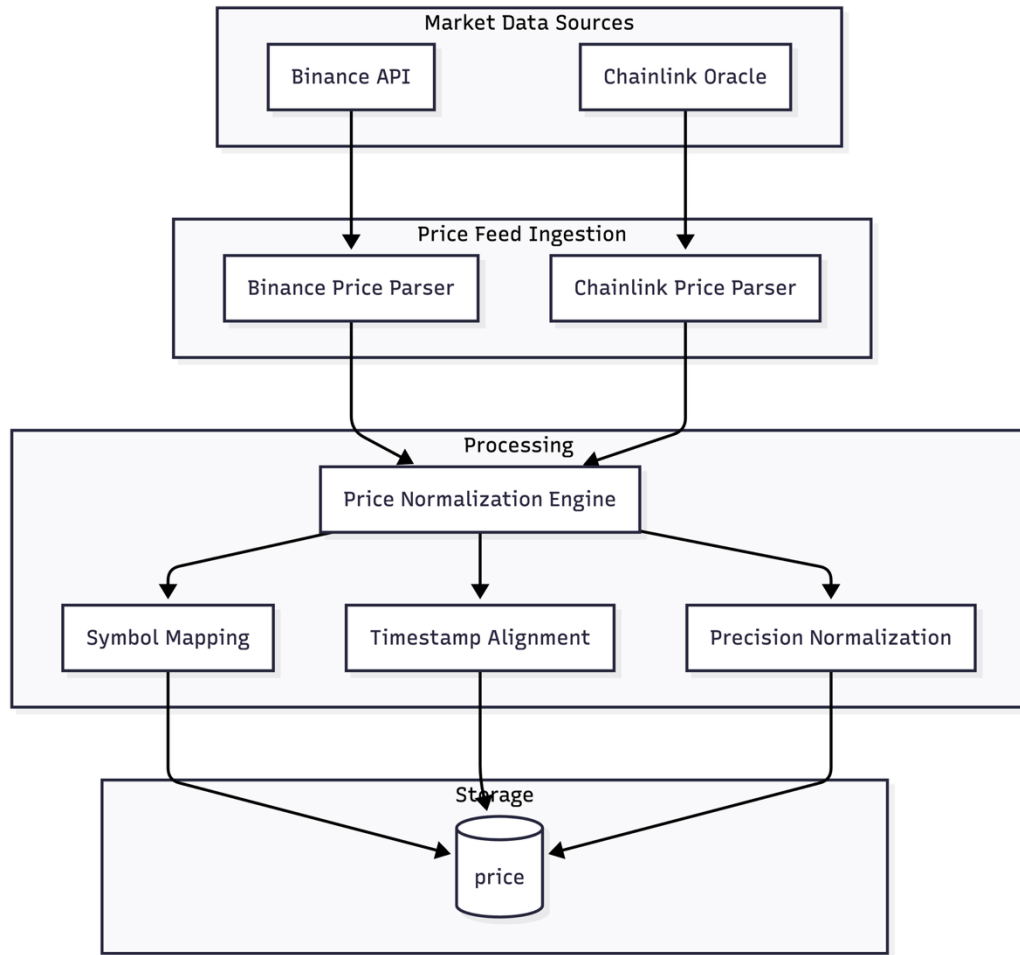


Architecture/pipeline

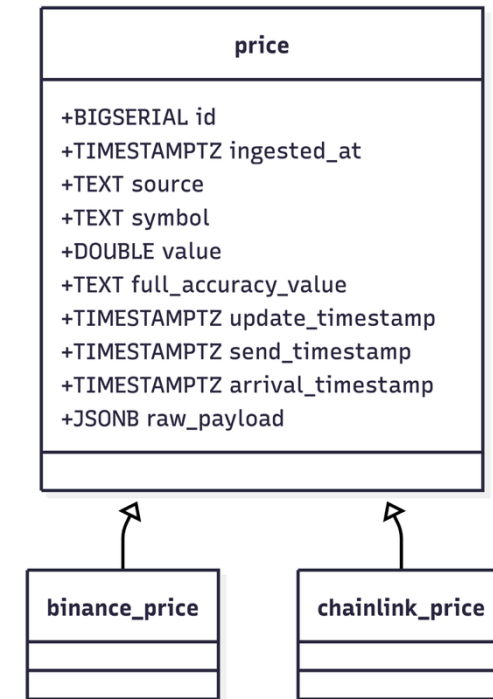


UML Class Diagram

UML Class Diagram - Price

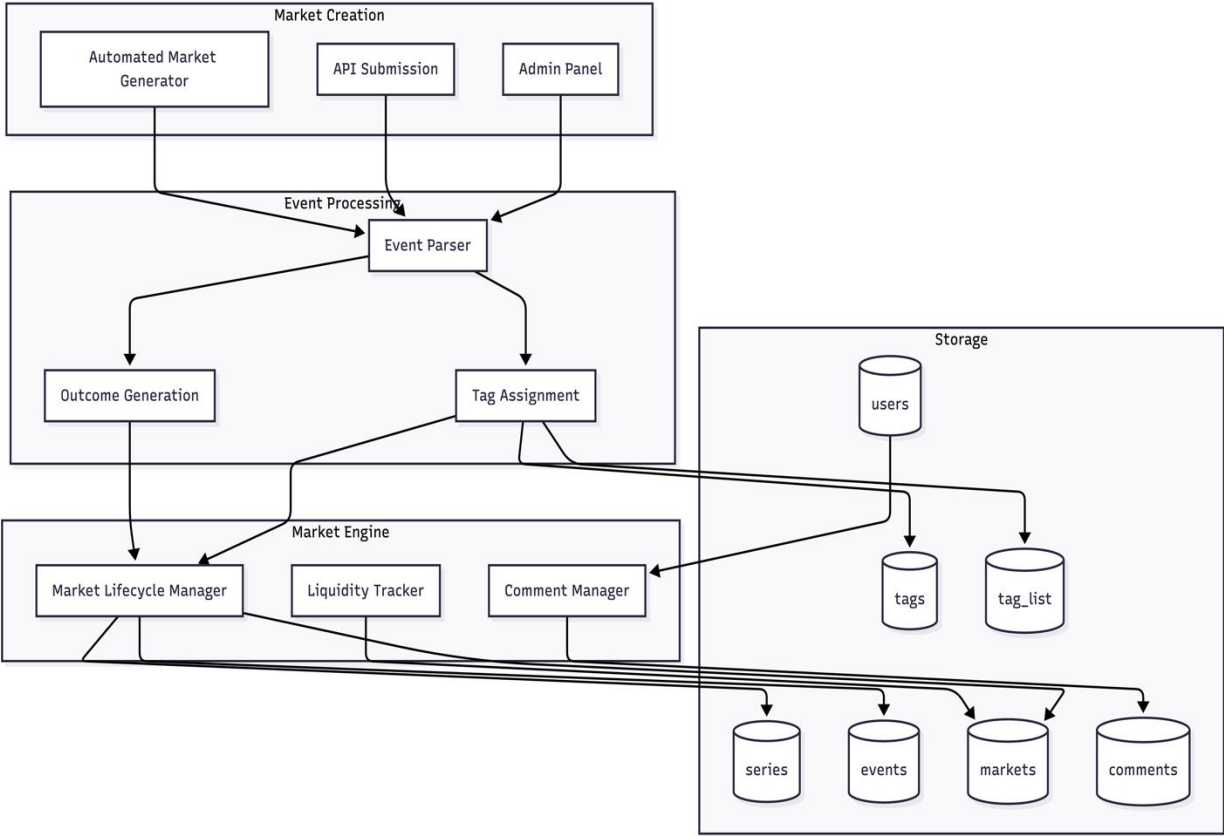


Architecture/pipeline

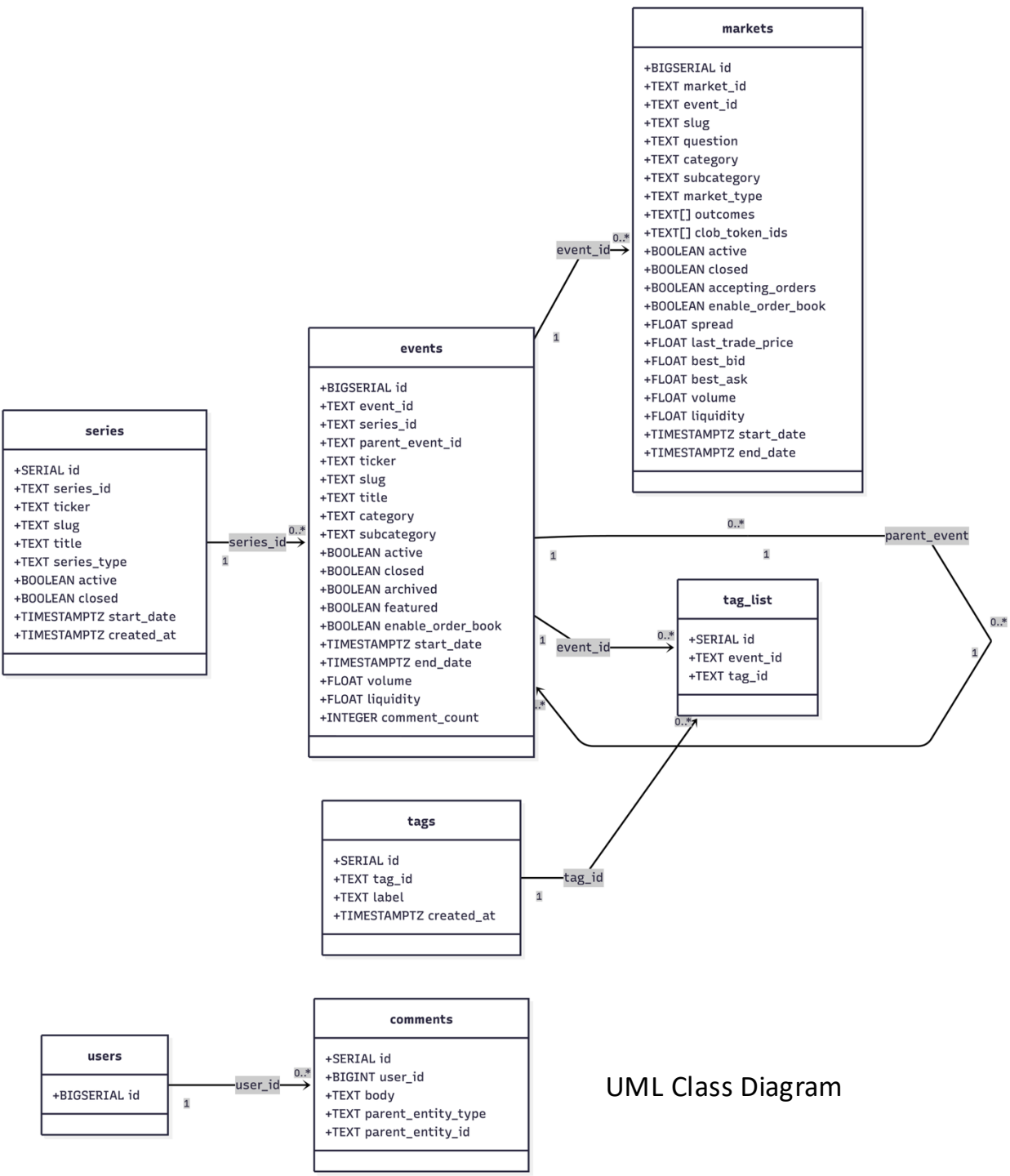


UML Class Diagram

UML Class Diagram - Market



Architecture/pipeline



UML Class Diagram

Architecture

2026/03/06 Design V1

Data source: Polymarket.

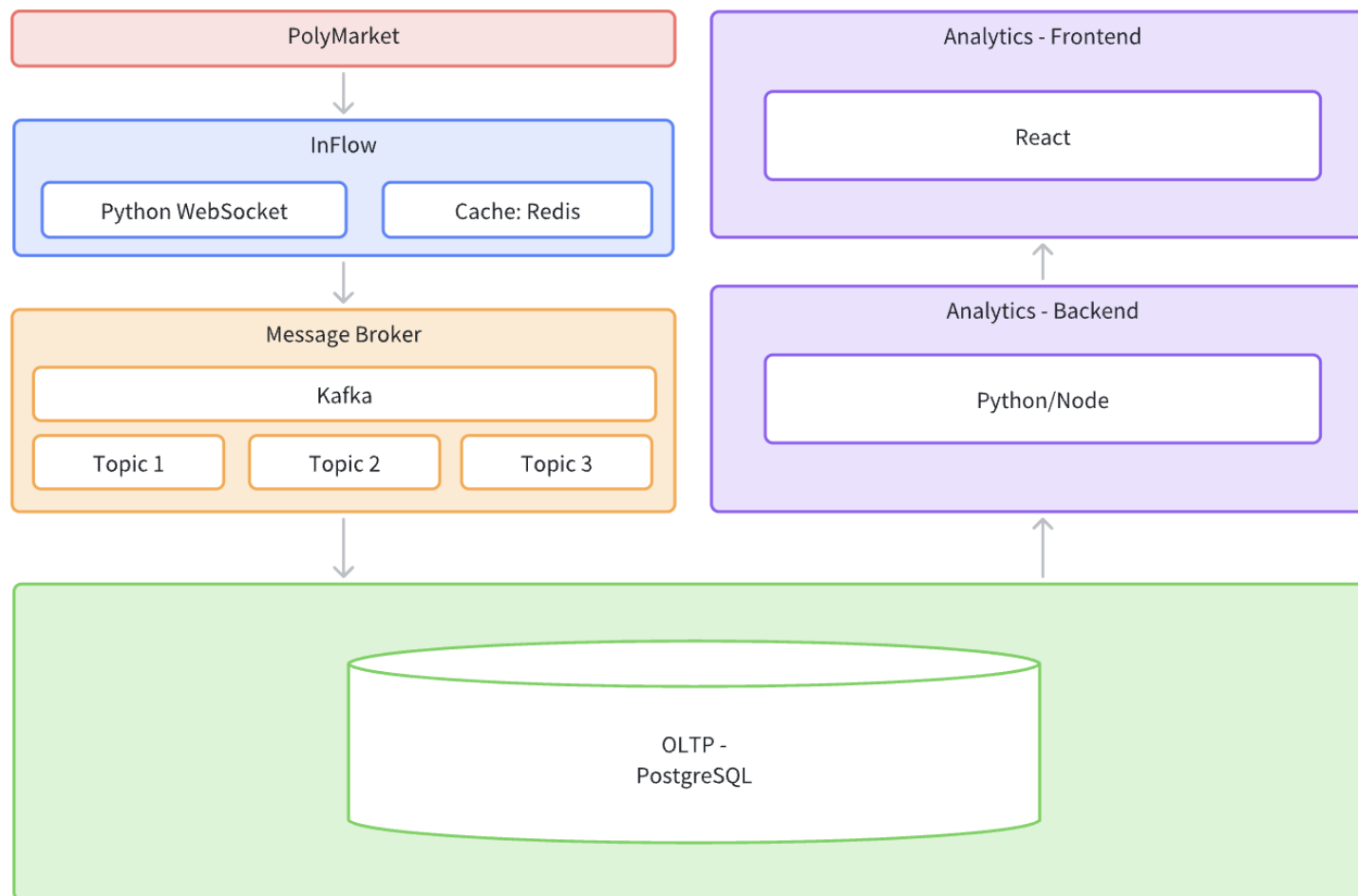
Ingestion layer: Python WebSocket collector with Redis cache.

Streaming backbone: Kafka message broker with multiple topics.

Storage: PostgreSQL (OLTP) for persisted normalized records.

Analytics backend: Python/Node services querying and aggregating stored data.

Analytics frontend: React UI for dashboards and research access.



Class design

Trading/Order Book System

- Core class: token, order_book_snapshot, order_book_update, trade, user
- Relations:
 - Token -> order_book_snapshot
 - Token -> order_book_update
 - Token -> trade
 - User -> trade

Price Oracle System

- Core class: price, binance_price, chainlink_price
- Relations: price --> binance_price / chainlink_price

Market Structure System

- Core class: series, events, markets, tags, tag_list, comments, users
- Relations:
 - series -> events -> markets
 - events -> events (parent_event)
 - events -> tag_list -> tags
 - users -> comments

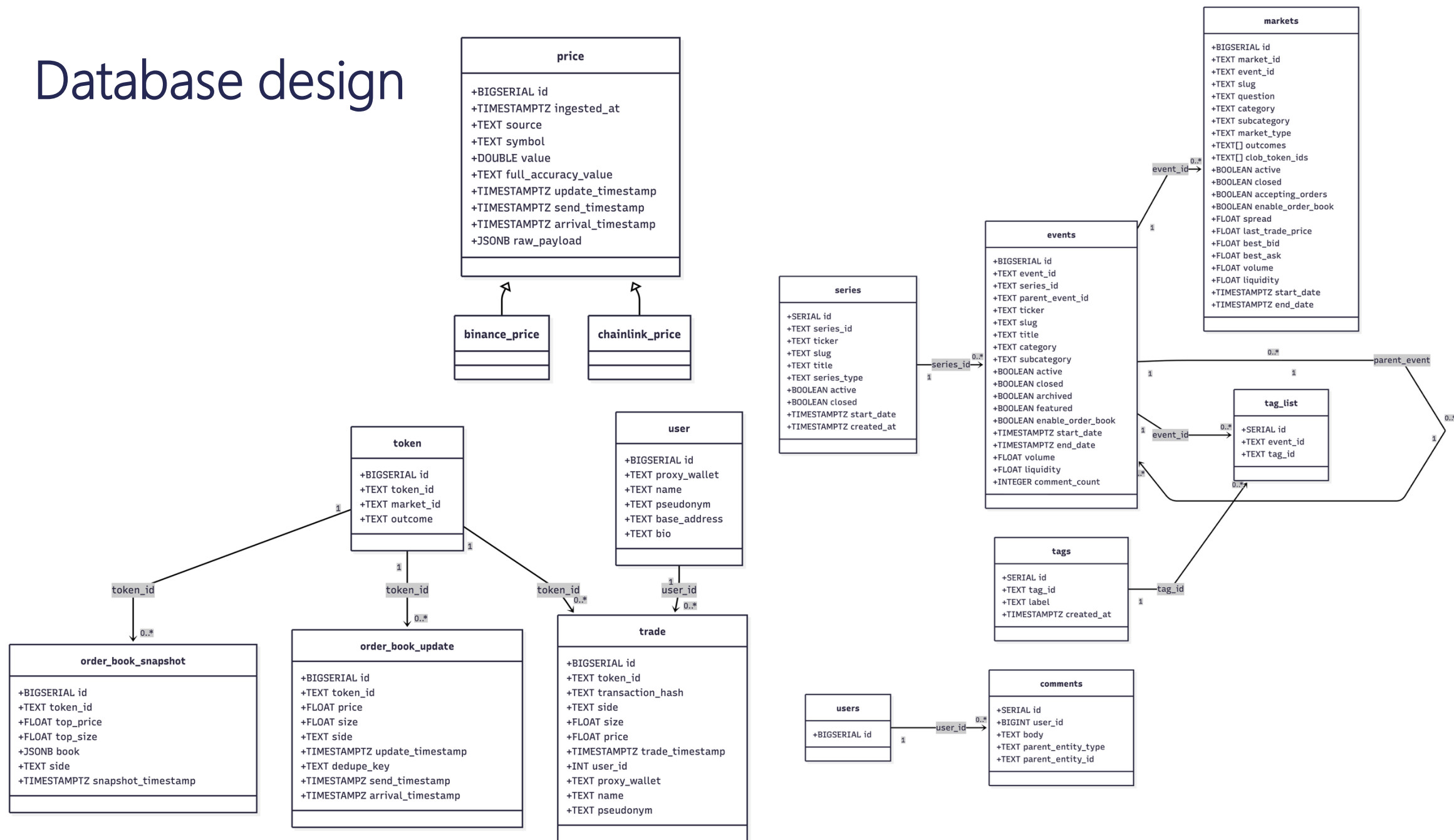
Usage

Store the order book state and trade executions for each token.

Provides external reference prices from different oracle sources.

Defines the hierarchical structure of prediction markets and their metadata.

Database design



APIs

External Polymarket Gamma REST endpoints documented:

- `GET /events`
- `GET /series`
- `GET /trades`

Internal persistence endpoint:

- `POST /internal/postgres/write`
- Allowed table targets:
- `chainlink\_prices`
- `binance\_prices`
- `kafka\_price\_changes`

Please refer to api.yaml file in the git repository for additional details.

Polymarket Gamma API 1.0.0 OAS 3.0

External API for Polymarket market data.

Servers

https://gamma-api.polymarket.com - REST API server

Filter by tag

default

GET /events Fetch market events

GET /series Fetch market series

GET /trades Fetch trade history

POST /internal/postgres/write Batch write to Postgres

Schemas

GammaEvent >

Testing approach Methodology

Unit Tests (The Foundation):

Testing individual functions in isolation.

Task: Does this specific Python function correctly parse a Polymarket probability format into a float?

Integration Tests (The Middle):

Testing whether decoupled system modules communicate correctly.

Task: Does the WebSocket ingestion service correctly write data to the cache?
Does the message broker correctly ingest and forward the event?

Data Quality & Contract Tests (Pipeline Specific):

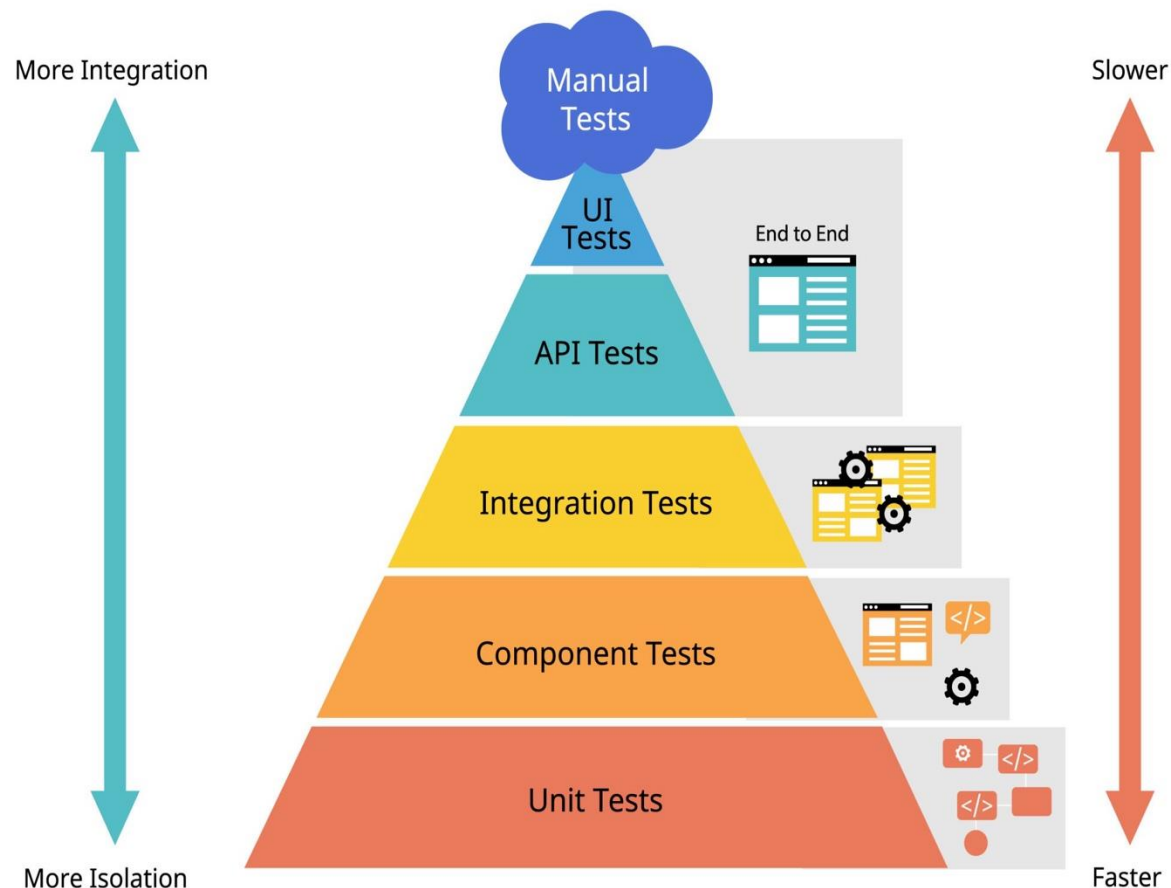
Testing whether schema or upstream data changes are handled safely.

Task: If Polymarket adds a new field to their JSON response, does our pipeline safely ignore it or crash?

End-to-End (E2E) Tests (The Peak):

Testing whether the full system works correctly from input to final user-visible output.

Task: If a trade event happens on the mock Polymarket WebSocket, does it successfully propagate through the pipeline, get stored in OLAP database, and appear correctly on the dashboard?



Testing approach In Data Ingestion

Corner Cases

Network Jitter & Disconnects.

What happens if the Polymarket WebSocket drops packets or silently disconnects? Does the system recognize the stale state?

Malformed Payloads.

Polymarket sends an unexpected null value for a betting pool size, or a string instead of an integer.

Redis Memory Eviction.

If the downstream pipeline slows down and Redis fills up, do we drop new data or block the WebSocket?

Examples from unit test to E2E test

Data Parsing

Objective: Verify that raw JSON from Polymarket is correctly transformed into our internal Python Data Class / Pydantic model.

Implementation: We do not need an internet connection for this. We save a sample Polymarket JSON response as a local fixture file. We write a pytest that feeds this static JSON into our parser function.

Python InFlow to Redis

Objective: Verify that parsed data is successfully cached in Redis with the correct Time-To-Live (TTL).

Implementation: We use Testcontainers (a framework that spins up a lightweight, throwaway Docker container of Redis just for the test). Our Python code connects to this temporary Redis instance, writes a mock message, and reads it back.

The Ingestion Slice

Objective: Verify the entire Inflow module works from WebSocket to Message Broker.

Implementation: We create a mock WebSocket server that mimics Polymarket and emits pre-determined trade events at a specific rate. We spin up our Python InFlow service, a Redis instance, and a single Kafka node using Docker Compose.

Error handling

Domain & Logic Safeguards (Task-Driven)

Malformed Ingestion Payload (Schema Changes)

Mechanism: Dead Letter Queue (DLQ). Catch `ValidationError`, route bad data to a separate Kafka topic, and skip to the next message without crashing the pipeline.

Quantitative Calculation Exceptions (e.g., Missing Data, `ZeroDivisionError`):

Mechanism: Graceful Degradation. Wrap core math in try-except blocks. Output `stale_data` flags or safe previous values to prevent NaN or Infinity from polluting the Clickhouse OLAP.

Infrastructure & System Safeguard (NFR-Driven)

Downstream Service Timeout (>500ms API/DB response)

Mechanism: Circuit Breaker Pattern. Temporarily halt requests to the failing service to prevent cascading failures; serve historical fallback data from the Redis cache.

Kafka/WebSocket Connection Drops

Mechanism: Exponential Backoff with Jitter. Catch `ConnectionError` and reconnect with increasing, randomized delays (e.g., 1s, 2s, 4s + random ms) to avoid network storming.

Spike in Trade Volume (Consumer Lag)

Mechanism: Backpressure & Auto-Scaling. Monitor Kafka consumer lag continuously. If thresholds are breached, trigger AWS ECS to dynamically spin up additional Python processing containers.

Non-Functional Requirements

Real-Time Performance

Low-latency ingestion and processing for live Polymarket data, enabling timely analytics and strategy response.

Reliability & Fault Tolerance

Automatic retry, reconnection, buffering, and graceful degradation to keep the pipeline running under partial failures.

Scalability

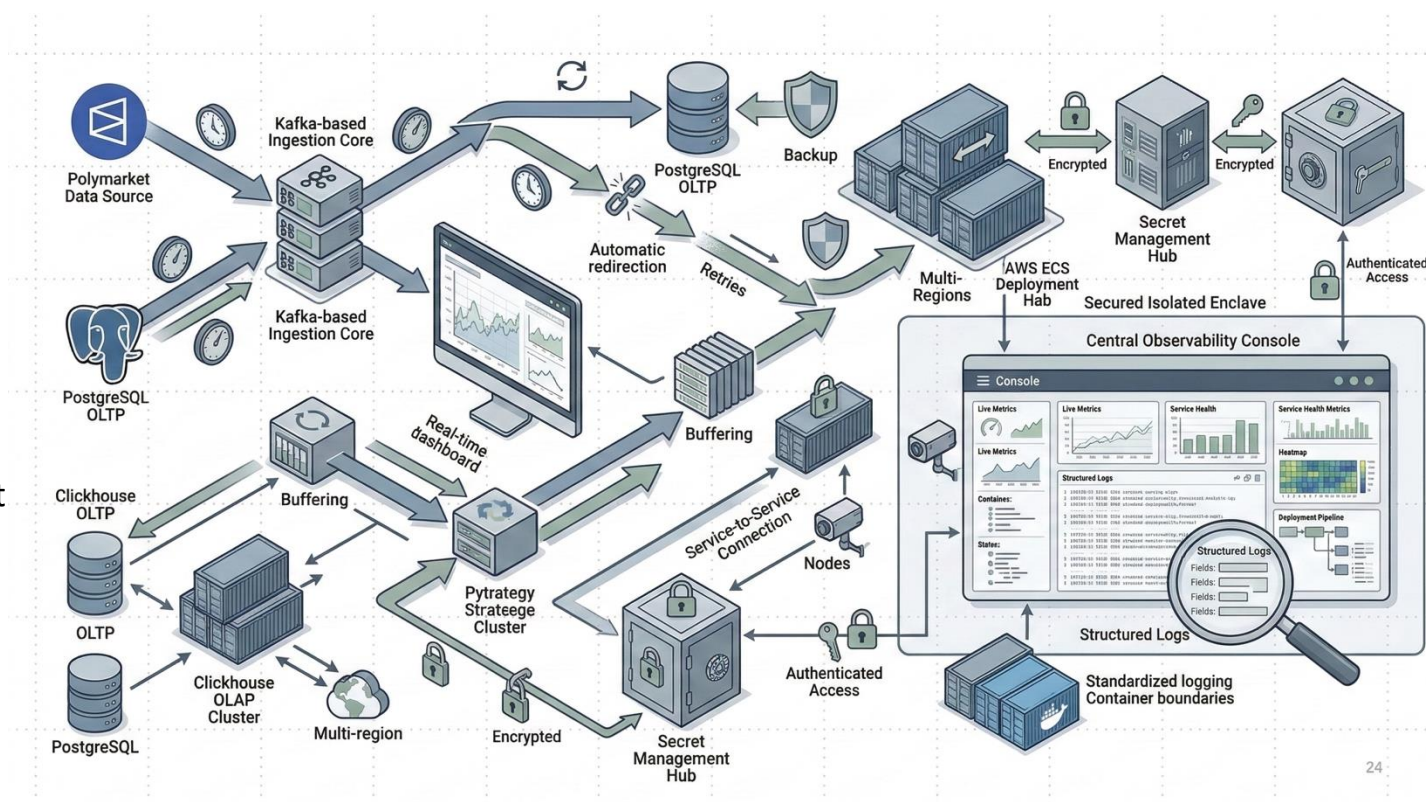
Modular services and message-driven architecture allow independent horizontal scaling as market volume grows.

Security & Access Control

Secure secret management, authenticated service access, and environment isolation to protect system resources and credentials.

Maintainability & Observability

Clear module boundaries, containerized deployment, structured logging, and monitoring to simplify debugging, testing, and future extension.



Status, Review and Planning

TA Discussion and Weekly Meetings Scheduled

We successfully met with our assigned TA on March 06 at 11:30 to review our project scope and requirements

We successfully scheduled with our assigned TA (Dongjie Ma) to weekly meetings on Friday at 12:30

The screenshot shows a Microsoft Teams meeting in progress. The video feed at the top displays three participants: Dongjie Ma, Loki Ziyi, and another person. Below the video, a browser window is open to the 'Duke' course page for 'Spring 2024'. The page features a 'Sprint 1 Rubric' table with columns for 'Criteria', 'Ratings', and 'Pts'.

Criteria	Ratings	Pts
Team Project GitHub Repository Repo for team project has been created	10 pts Full Marks	0 pts No Marks
Project Requirements 1. Create a lean canvas document for your project. 2. Develop two or more paper prototype screens to highlight the significant functionality of your application. 3. Discuss your project with your assigned TA	15 pts Full Marks	0 pts No Marks
Project Backlog Create a set of GitHub issues representing the requirements you've identified for the project.	10 pts Full Marks	0 pts No Marks
Initial Design Perform the initial design for your application. Items to consider: Architecture Class design Database design	10 pts Full Marks	0 pts No Marks
APIs Testing approach Error handling Implementing necessary non-functional requirements (security/performance)	10 pts Full Marks	0 pts No Marks
Weekly Meetings Scheduled Schedule your team's weekly meetings with your assigned TA.	5 pts Full Marks	0 pts No Marks
Status Report Each project team must complete a sprint status report at the end of each spring with the following elements: A list of the GitHub issues opened or closed during the week. UML class diagram (The first report will be the most difficult; later reports will need to update the diagram.) A paragraph summarizing your sprint review A paragraph summarizing your sprint retrospective A paragraph summarizing your sprint planning Any additional comments and/or concerns	15 pts Full Marks	0 pts No Marks

The screenshot shows a Microsoft Teams calendar event titled "512 - Weekly Meetings - Team Hotel". The event is scheduled for Friday, March 20, 2026, from 12:30 PM to 1:00 PM. The event details include the meeting ID, password, and a link to join the meeting.

Calendar (rh398@duke.edu)

512 - Weekly Meetings - Team Hotel

Participants: Dongjie Ma, Mory Shi, Haoan Lin, Loki Ziyi, Yinglun Song, Rebeca Hechtman

Time: 3/20/2026 from 12:30 PM to 1:00 PM

Repeat: Repeat weekly Occurs every Friday effective Mar 20, 2026 until May 2, 2026 from 12:30 PM to 1:00 PM.

Microsoft Teams Meeting

Join: <https://teams.microsoft.com/meet/24650205388281?p=2C2HJ4eS9mkAO3H6m6>

Meeting ID: 246 502 053 882 81

Password: v3T7hC7e

For organizers: Meeting options

Sprint Review Summary

During this initial sprint, our primary focus was on establishing the foundational product strategy, defining core requirements, and setting up our technical infrastructure for our core product.

We successfully finalized the Lean Canvas, which aligned the team on the application's value proposition and target audience. Additionally, we developed some of our initial paper prototypes to visualize the core user flow and held our kickoff discussion with our assigned TA to validate our approach. The initial architecture and design frameworks, such as have been established, setting a solid groundwork for upcoming development.

From a technical perspective, we set up the GitLab repository and organized the project backlog through structured issues to guide development tasks. The team also produced initial system design artifacts, including UML class diagrams for the order book, price oracle, and market structure modules, as well as a preliminary architecture for the data pipeline.

Sprint Retrospective Summary

Looking back at Sprint 1, the team collaborated effectively to bridge the gap between business requirements and technical design. Our communication was strong, particularly when brainstorming the project and flow. However, we also observed that some technical alignment—particularly around API structures, data ingestion design, and the testing strategy—took longer than expected. These discussions required deeper coordination because multiple components of the system depend on them. For Sprint 2, we plan to improve efficiency by breaking complex architectural decisions into smaller discussion topics earlier in the week and assigning clearer ownership for technical components. This will help the team reach consensus faster and maintain steady development progress.

Sprint Planning Summary

For Sprint 2, our objective shifts from initial planning and strategy into active development and implementation. We plan to tackle the highest-priority items currently in our project backlog. Key deliverables for the upcoming week include

1. initializing the database,
2. define the unit tests around core functions, and
3. having the ETL pipeline up and running.

We have scheduled our recurring weekly TA meeting for Friday 12:30 to ensure we remain aligned with the project constraints and technical requirements.

Additional Comments and/or Concerns

General Comments: The team is aligned and ready to begin executing the technical backlog in the upcoming week.

Risks/Blockers: Currently, there are no major blockers. However, we will be actively monitoring concerns as the ones below:

Risk1 - API Constraints: We are closely monitoring the Polymarket Gamma REST and WebSocket endpoints. Handling potential API rate limits, invalid responses, and maintaining gapless ingestion will be a primary technical hurdle as we build the "Hot Path".

Risk2 – API Access Concern: Polymarket may retrieve/change the accessibility to their official API endpoints, and it will significantly impede our development.

Risk 3 – Prediction Model Performance Risk: The performance of the prediction model may not meet the required accuracy or reliability thresholds. If the model fails to generate meaningful predictions, the analytics component may not provide sufficient value for deployment in the production dashboard.

Thank you!